

7. Text Analytics

Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization. 2. Create representation of document by calculating Term Frequency and Inverse Document Frequency.

```
import nltk
```

```
nltk.download("all")
```

```
from nltk import word_tokenize, sent_tokenize
```

```
corpus = "Sachin was the GOAT of the previous generation. Virat is the GOAT of this generation.  
Shubham will be the GOAT of the next generation."
```

```
print(word_tokenize(corpus))
```

```
print(sent_tokenize(corpus))
```

```
from nltk import pos_tag
```

```
tokens = word_tokenize(corpus)
```

```
print(pos_tag(tokens))
```

```
from nltk.corpus import stopwords
```

```
stop_words = set(stopwords.words("english"))
```

```
#print(", ".join(list(stop_words)[:]))
```

```
stop_words
```

```
tokens = word_tokenize(corpus)
```

```
cleaned_tokens = []
```

```
for token in tokens:
```

```
    if (token not in stop_words):
```

```
        cleaned_tokens.append(token)
```

```
print(cleaned_tokens)
```

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
stemmed_tokens = []
```

```
for token in cleaned_tokens:
```

```
    stemmed = stemmer.stem(token)
```

```
    stemmed_tokens.append(stemmed)
```

```
print(stemmed_tokens)
```

```
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()
```

```
lemmatized_tokens = []
```

```
for token in cleaned_tokens:
```

```
    lemmatized = lemmatizer.lemmatize(token)
```

```
    lemmatized_tokens.append(lemmatized)
```

```
print(lemmatized_tokens)
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from nltk.probability import FreqDist
```

```
corpus = [  
    "Sachin was the GOAT of the previous generation",  
    "Virat is the GOAT of the this generation",  
    "Shubham will be the GOAT of the next generation"]
```

```
vectorizer = TfidfVectorizer()
```

```
vectorizer.fit(tokens)
```

```
fdist=FreqDist(tokens)
```

```
for word,frequency in fdist.items():
```

```
    print(f"{word}:{frequency}")
```

```
tfidf_matrix = vectorizer.transform(tokens)
```

```
print(tfidf_matrix)
```

```
print(vectorizer.get_feature_names_out())
```